Homework 3

Due date: July 20, 23:59

General Guidelines:

1. **Watch the online tutorial for introduction to this homework (located in the Moodle)**
2. You should use only C++11 for this homework.
3. All your code should be compiled with "-O3" g++ flag (maximum optimizations).
4. Make sure to create new classes and files with the exact names defined ahead. Otherwise, you will face compilation errors during automatic testing.
5. This homework assignment includes several .h files. Read ahead.
6. Don't use any external libraries (not part of C++ standard), except TBB, for this homework.
7. The homework assignment is fully auto graded. You can resubmit your solution to gradescope multiple times before the due date of this homework assignment.
8. The automatic tests for this homework run for **few minutes**, per submission.!!
9. The homework includes a bonus of 25 point. You can earn up to 125 points for this exercise.
10. In total, you should upload (together) two files to gradescope:
    a. nbody_impl.h (exercise #1)
    b. skiplist_par_impl.h (exercise #2)

Exercise 1: Parallel Nbody simulation – 62 points

The Nbody discrete simulation simulates the movement of particles in a multi-dimensional space due to gravity forces between every two particles (https://en.wikipedia.org/wiki/N-body_problem). It can naturally be parallelized over sequential implementations.

You are given a fully sequential implementation of the Nbody simulation in 3D space. Your goal is to develop the most efficient parallel implementation for this simulation.

1. You are given *nbody.h* which includes three functions with the suffix of *"_serial". These* are the building blocks for the sequential Nbody simulation. Don't change the sequential code!
2. This *nbody.h* also includes the signatures of three equivalent functions (with the "_parallel" suffix) that you need to implement for the parallel version.
3. You should create *nbody_impl.h* (make sure to "*#include "nbody.h")* with the implementation of these missing *"_parallel"* functions.
4. You can add data structures and help functions to *nbody_impl.h, as needed.*
5. You are also given *main_nbody.cpp*. It is the main code we use in gradescope to measure the speedup of your parallel implementation. It can help you with the development, debugging and measuring the speedup of your parallel implementation.
6. Don't try to submit *main_nbody.cpp* to gradescope. **The only source file you should eventually submit to gradescope is *"nbody_impl.h".***

The following are some additional guidelines for development:

- Overall, learn all existing code carefully before you start to develop the parallel version.
- Your parallel implementation should be based on both multi-threading and vectorization (you can assume AVX2 support in gradescope).

- To ensure the correctness of your parallel code, you should initialize the particles in your parallel implementation with the exact values used in the reference sequential code. Otherwise, the correctness tests in gradescope will fail.
- If you plan to directly use AVX2 to vectorize your code, make sure to *"include <immintrin.h>"* inside your code and use the additional "*-mavx2"* command-line flag (if you compile locally). (examples of using AVX2 included in the course material on vectorization).
- References to intrinsic ops include:
  - https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html (press the right checkbox for vectorization ISA of your machine e.g., AVX2.)
  - An example of vectorized code with intrinsic op in the lecture presentation titled *"multicore processors -part I"*
- Due to dynamic resource allocation in gradscope, your parallel version should be independent of a specific number of threads. Hence, you should use only the TBB library for multi-threading (No C++ threads)
- You can assume Intel cores with L1 cache size=32KB and cache line size=64B.
- You are NOT forced to reuse code from the serial implementation for your parallel version. Your parallel version (the functions with "__parallel" suffix) can be developed completely from scratch.
- See additional guidelines given in the online tutorial for this homework regarding the submission and grading of this exercise.

Exercise 2: concurrent skiplists – 63 points

Implement a concurrent skip-list. Your implementation should use the same interface as the sequential skiplist from HW0 (the *skip_list_seq.h* file).

- Your parallel version should be based on the idea of hand-to-hand locking
  - See the tutorial on fine-grain synchronization for concurrent linked-list
- Parallelism should be supported at every level of the skiplist.
- Remove ops should directly delete nodes.
- See about std::recursive_mutex in the online tutorial for this homework

Implement a class *SkipListImpl* with your parallel implementation. This class should inherit the abstract class *SkipList* in *skip_list_seq.h*.

Submit the *skiplist_par_impl.h* file to gradescope with your implementation of the *SkipListImpl* class.

Good luck!